

Video by techwithnt

Transcribed by [TurboScribe](#). [Go Unlimited](#) to remove this message.

AI observability is becoming important because an AI app can fail even when nothing technically breaks. Your server can still be healthy, your API can be fast, your dashboard can be all green and your chatbot can still give a completely wrong answer to a real customer and you would never know. This is exactly why AI observability is becoming one of the most important ideas in LLM engineering.

Hi, I'm Nitya and this is day 15 in our 50-day series of understanding AI in 2026. So in normal software, when something breaks, it is usually obvious. The page crashes, the API call gets slow or the database throws an error.

But LLM apps or AI apps fail in a much weirder way. The app can be online, the response can be fast and the answer can still be completely wrong. So AI observability or LLM observability means tracking what happened inside your AI system from the moment the user asked a question to the moment the final answer came out.

And the most important technical word here is called a trace. A trace is a full step-by-step journey of one AI request. So imagine you own an online business and your customers asks your chatbot about a refund policy.

Something like, can I return this item after 30 days? And the bot hallucinates a completely wrong answer and it says yes, you can return the product after 30 days. Without observability, the team may never notice that one specific customer question and that one wrong bot response. They only find out later when the customer leaves a frustrated review saying that they waited 30 days, tried to start the return and the refund option was no longer available on the website.

But with observability rules inside the AI system, that response can be flagged the moment it looks unsupported, risky or inconsistent with the company policy. Then the team can open the trace and see exactly what happened. Something like, did the retriever pull the wrong policy document? Or did the model ignore the right document? Or did the prompt version change yesterday? Or did the agent call the wrong tool? So all of these answers can be answered when you look at a trace.

So monitoring basically asks, is the system alive? And observability asks, why did the system behave this way? Why did the system give a wrong answer to the customer? And for Gen AI systems, that second question matters more than anything. Because in AI products, quality is not just uptime. Quality means the answer is useful, grounded, safe, not hallucinated, and not too expensive.

And not randomly worse after one prompt change. And that's how important AI observability is in

2026. So if you want to build a practical project around this, build a tiny rag chatbot with 20 documents, add an observability dashboard with a defined set of rules.

So you can track every question, track the retrieved chunks, track the model response, track latency, tokens, cost, answer relevance, and user feedback. And then create one page called bad answers. Every time the bot gives a weak answer, doesn't follow the rules, open the full observability trace, find the failure point, fix the prompt or retriever, and test again.

And that is how real AI products can improve. And moving on to the most interesting part is the question for today. In AI observability, what is the full step-by-step record of one AI request called? And here are the options.

Please write your answers in the comments below. And I can't wait to talk to you all in the next episode.

Transcribed by [TurboScribe](#). [Go Unlimited](#) to remove this message.